

A Comprehensive Analysis of CPU Allocation Fairness in the Multi-Level Feedback Queue (MLFQ) Algorithm

Multi-Level Feedback Queue (MLFQ) scheduling algorithm is an advanced and extensively implemented algorithm used to tradeoff low interactive response time and high overall system throughput. Nonetheless, it is a dynamic and feedback-driven basis, thus, unintentionally, it can create major equity concerns in CPU sharing. This report will break down these fairness issues, with a concentration on the natural advantages that are being given to the I/O-bound processes, and the consequent risks of starvation that the CPU-bound processes face. Moreover, it presents and uses a quantitative measure, the Fairness Index, to assess fairness in scheduling in case of a sample workload, and explains the ways to alleviate identified imbalances.

1. Unfair CPU Allocation due to I/O-Bound Processes

1.1. Dynamic Priority Adjustment in MLFQ :

The idea behind the MLFQ is to have numerous queues in which the rule of thumb here is to have various queues with varying priorities and time quantum. The processes are first put in the highest-priority queue. The scheduler processes the highest non-empty queue. The dynamic prioritization process is regulated by two important principles under the observed process behavior:

1. Rule for CPU-bound processes: When a process consumes its time quantum (i.e. it does not release the CPU voluntarily at the expiry of the quantum), the process is deemed to be CPU-bound and is preempted to a lower-priority queue.
2. Rule for I/O-bound processes: When a process provides the CPU with a voluntary yield before the time quantum elapses (in most cases, to make an I/O request), the process is said to be interactive or I/O-bound. It is left to stay in that priority queue.

With this feedback mechanism, MLFQ makes it possible to approximate the Shortest-Job-First (SJF) scheduling through prioritizing the short interactive jobs.

1.2. I/O-Bound Processes Monopolization and CPU-Bound Processes Demotion :

Processes with I/O-bound periods, like text editor or shell consoles, intrinsically have short CPU bursts, then long I/O wait periods. Based on the MLFQ rules, such processes are always doing a very small portion of their quantum before giving the CPU the CPU on I/O. This leads to them not being punished with demotion and always staying in the top priority lines. They are scheduled almost immediately whenever they are ready (e.g. when a key is pressed) and so offer excellent responsiveness.

On the other hand, CPU bound processes such as video encoders or scientific simulations need extended uninterrupted CPU times. They continuously spend all their time quantum and are then demoted down the queue levels. Once within the lower queues, they get CPU time only when all the higher-priority queues have no higher-priority customers. This produces an imbalance in the structure wherein a handful of I/O-bound tasks are able to capture the high-priority levels.

1.3. Conclusion Starvation and Inequality :

The mentioned imbalance directly causes starvation or unfair allocation of CPU to CPU-bound processes. In case an ongoing stream of I/O-bound activity exists in the system, the low-priority queues can never be served. A CPU bound process that is moved to the lowest queue may end up being blocked permanently even when it is ready. This is in contravention of the basic objective of schedules in fairness whereby all processes must make reasonable progress.

1.4. Fairness-Improving Mechanisms:

To address these problems of unfairness inherent to it, there are a number of corrective mechanisms used in contemporary implementations of the MLFQ:

Aging: This is one of the basic methods of forestalling starvation. The scheduler periodically raises the priority of the processes that have been waiting in the low-priority queues too long. As an example, a process can be advanced a step higher every few seconds. This will ensure that a CPU-bound job will eventually get augmented to a higher priority, and be allocated more CPU time and progress.

Example: The compilation of a long-lived gcc may be starved out without aging. On aging, it is demoted to medium-priority queue after a wait of 10 seconds at the lowest queue, and gets an opportunity to run.

Priority Boosting: Every (e.g. every T milliseconds) the scheduler preempts all processes in the system to the topmost queue. This process re-initiates the history of the scheduler and makes sure that no process, no matter what its history is, is permanently starved. It enables CPU-bound jobs to restart all over again and compete fairly with interactive ones in regards to CPU time.

Example: A priority boost every one second would interrupt long CPU-bound process waits, giving them a slice of high-priority CPU time at a regular rate that would vastly enhance fairness.

Time Quantum Adjustments: Time quantum per queue is a very important parameter. Queues with higher priority usually have lower quantum (e.g. 8-16 ms) to support interactive jobs. Queues with lower priority have larger quantum (i.e., 64-256 ms) to minimize the overhead associated with context-switching among jobs that take up a lot of CPU time. A trade-off between responsiveness and fairness can be made by regulating these values. This is because an excessively short quanta in lower queues may be detrimental to throughput whereas an excessively long quanta may raise the response time of the newly arrived interactive jobs.

2. Quantitative Fairness Evaluation

2.1. Defining Fairness in CPU Scheduling :

When it comes to CPU scheduling, fairness is the term used to describe the fair allocation of the CPU resource to all the competing processes. A fair scheduler guarantees that there is no unfair preference or starvation of any process and all the jobs move proportionately to their needs and priority. Multi-queue systems such as MLFQ have a special problem of fairness since the incentive of responsiveness (putting a priority on I/O-bound jobs) is usually incompatible with the incentive of fair allocation.

2.2. The Fairness Index Metric :

Jain Fairness Index is a common quantitative index of fairness. This index gives one number between 0 and 1 with 1 being the utmost fairness. It is characterized as follows in case of a group of n processes:

$$Fairness\ Index = \frac{(\sum_{i=1}^n T_i)^2}{n \cdot \sum_{i=1}^n T_i^2}$$

Where T_i is the total CPU time allocated to process i .

2.3. Applying the Metric to a Sample MLFQ Workload :

Based on a workload, the three processes are scheduled to a fixed time with an MLFQ scheduler, and there are two queues (Q1: high priority, quantum=10ms; Q2: low priority, quantum=40ms). No aging or priority boosting: assume that in this first analysis, there is no aging or priority boosting.

- IO Process A (I/O-bound): 5ms bursts and then does I/O.
- Process B (CPU-bound): Long, continuous CPU demand.
- Process C (CPU-bound): Long, continuous CPU demand.

Hypothetical Outcome: Within the period of measurement, the accumulated CPU time may be:

- $T_A = 60\ ms$ (frequently scheduled in Q1)
- $T_B = 25\ ms$ (mostly scheduled in Q2)
- $T_C = 15\ ms$ (mostly scheduled in Q2)

Calculating Fairness Index:

$$\sum T_i = 60 + 25 + 15 = 100$$

$$\sum T_i^2 = 60^2 + 25^2 + 15^2 = 3600 + 625 + 225 = 4450$$

$$Fairness\ Index = \frac{(100)^2}{3 \times 4450} \simeq 0.75$$

This value of 0.75 corresponds to a high amount of unfairness, which is consistent with our qualitative analysis: the I/O-bound process of (A) received a disproportionate amount of CPU time compared to the CPU-bound processes of (B) and (C).

2.4. Analyzing the Impact of MLFQ Parameters on Fairness :

- **Number of Queues:** When the number of queues is increased, it may further increase unfairness by giving the opportunity to separate the highest priority and the lowest priority level. A process that is demoted to the bottom of a 5-queue system cannot easily receive CPU time as compared to a 2 queue system..
- **Quantum Lengths:** By reducing the quantum in shorter queues, stalling ones serve to exacerbate the performance of CPU-bound jobs by incurring higher context-switch overheads as well as decreasing the performance of their allocated slices. The extension of the high-priority quantum may enable the I/O-bound jobs to act more like CPU-bound jobs, and hence they may be demoted and more fair to others.
- **Aging Intervals:** the most direct positive influence on the Fairness Index is the introduction or reduction of the aging interval. In our sample workload, when the process B and C were aged to Q1, periodically, would increase, their $T_A = 50$, $T_B = 30$, $T_C = 20$, the new Fairness Index would be:

$$\frac{(100)^2}{3 \times (2500 + 900 + 400)} = \frac{10000}{11400} \simeq 0.88$$

This demonstrates a clear improvement in the quantitative fairness score.

3. Conclusion

MLFQ algorithm is a strong heuristic that can be used to optimize the performance of the system in an interactive environment. Nevertheless, its fundamental feedback structure by definition presents a fairness problem whereby it systematically favors I/O-bound processes to the detriment of the CPU-bound processes. This may result in extreme

starvation. Such quantitative measures as the Fairness Index created by Jain are useful in objectively assessing this imbalance. Incorporation of corrective measures like aging and priority boosting is not only an improvement but a requirement to make MLFQ work in a general-purpose operating system since it directly reduced unfairness and endless starvation, moving the Fairness Index toward the ideal value of 1.

4. References

1. Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2018). Operating Systems: Three Easy Pieces. Arpaci-Dusseau Books. (Chapter 8: Scheduling: The Multi-Level Feedback Queue).
2. Jain, R. K., Chiu, D.-M., & Hawe, W. R. (1984). A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems. Digital Equipment Corporation, Technical Report TR-301.
3. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley. (Chapter 5: CPU Scheduling).
4. Tanenbaum, A. S., & Bos, H. (2014). Modern Operating Systems (4th ed.). Pearson. (Chapter 2: Processes and Threads).